

✓ Welcome to Colab!

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call

```
#!pip install langchain
#!pip install langchain_groq
#!pip install langchain_community
#!pip install langchain_text_splitter
#!pip install langchain_chroma
#!pip install langchain_groq
```

```
import os
#from langchain.document_loaders import UnstructuredFileLoader #old import n
from langchain_community.document_loaders import UnstructuredFileLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_groq import ChatGroq
from langchain_classic.chains import RetrievalQA
```

```
/tmp/ipykernel_16030/1170151002.py:3: DeprecationWarning: `langchain-communit
from langchain_community.document_loaders import UnstructuredFileLoader
```

```
os.environ["GROQ_API_KEY"]='gsk_YN6BYzSEPlGcuUkoHz5WGdyb3FYVcpWb1jQl2gDix3e
```

```
import requests
url="https://rbberger.github.io/assets/files/smr2574/OOPConcepts.pdf"
response = requests.get(url)
with open('OOPConcepts.pdf', 'wb') as f:
    f.write(response.content)
```

```
#!pip install unstructured[pdf]
```

```
loader = UnstructuredFileLoader("OOPConcepts.pdf")
```

```
/tmp/ipykernel_16030/743537071.py:1: LangChainDeprecationWarning: The class `
loader = UnstructuredFileLoader("OOPConcepts.pdf")
```

```
documents=loader.load()
documents
```

```

// Will call Boeing747::fullThrottle\n\nb->fullThrottle();\n\n// NEW! will call
Cessna206h::fullThrottle\n\nc->fullThrottle();\n\nObjects of derived
classes can be used as base class objects.\n\nPolymorphism allows us to
modify the behavior of base classes and replacing the implementation of
methods. • Polymorphic methods must be\n\ndeclared virtual (in
C++)\n\nPolymorphism\n\n// store objects of different types in a //
datastructure using its base class Airplane ** airplanes = new Airplane*
[10]; ... for(int i = 0; i < 10; i++) {\n\n// will choose correct
implementation // dynamically at runtime airplanes[i]-
>fullThrottle();\n\n}\n\nvoid foo(Airplane & a) {\n\n// this function will
work with // any class derived from Airplane\n\n}\n\nUse base class to
implement general algorithms and data structures which work with any
derived type\n\nDynamic dispatch determines the type of an object at
runtime and executes the correct method\n\nAbstract classes\n\nclass
Airplane {\n\npublic:\n\nfloat longitude, latitude;\n\nfloat heading,
speed;\n\nfloat altitude;\n\nvirtual void fullThrottle() =
0;\n\n};\n\nVirtual functions without implementation are called pure-
virtual functions. • Classes containing pure- virtual functions can not be
instantiated and are called abstract classes\n\nTheir behavior must be
defined in derived classes\n\nIs-A vs. Has-A Relationship\n\nUse
composition to manage complexity • Use inheritance to extract common
functionality • Use polymorphism to implement general algorithms which are
independent of specific types\n\nComposition = Has-A Relationship E.g. A
Boeing 747 has four jet engines.\n\nInheritance = Is-A
Relationship:\n\nE.g. A Cessna is an airplane. So is a Boeing 747.\n\nIII.
Best Practices\n\nRecommendations, Design Patterns\n\nKeep your interfaces
simple, clean and consistent • Methods in a class should be as orthogonal
as possible\n\navoid method that do almost the same\n\nMethods with
the same name should do similar things • Use encapsulation\n\nThis reduces
coupling • Changing your implementation internals becomes easier • Access
data with getter / setter methods\n\nUse const to limit what methods can do
with your data\n\nThis lets the compiler help you enforce who can
manipulate data • Compiler Optimization hint\n\nConsider using lazy-
initialization for costly object properties\n\nGetters & Setters\n\nclass
SomeClass { int propertyA;\n\nconst method does not modify
data\n\npublic:\n\nint getPropertyA() const {\n\nGetter function\n\nreturn
propertyA;\n\n}\n\nvoid setPropertyA(int value) {\n\nSetter function\n\n//
setters allow you to validate data // before accepting it if (value > 0 &&
value < 100) {\n\npropertyA = value;\n\n}\n\n}\n\nLazy
Initialization\n\nclass SomeClass {\n\nLargeDataSet *
dataSet;\n\npublic:\n\nSomeClass() : dataSet(nullptr) { }\n\nDon't create
data set during construction\n\n~SomeClass() {\n\ndelete dataSet;\n\nWon't
do anything if data set is never created\n\n}\n\nint getDataSet() { if
(!dataSet) {\n\ndataSet = new LargeDataSet();\n\nCreate data set set on-
demand\n\n} return dataSet;\n\n}\n\n}\n\nBase class destructors should be
virtual\n\nIf you want to allow deletion of objects by using their base
class, make sure their destructor is virtual\n\nclass Base {
public:\n\nclass Base2 { public:\n\n~Base();\n\nvirtual ~Base2()
{}\n\n}\n\n}\n\nclass Derived : public Base { public:\n\nclass Derived2 :
public Base2 { public:\n\n~Derived();\n\nvirtual ~Derived2()
{}\n\n}\n\n}\n\nBase * object = new Derived(); delete object; // undefined
behavior!!! // best case: only ~Base() is called // potential memory
leak\n\nBase * object = new Derived(); delete object; // first ~Derived()
is called, // then ~Base()\n\nDon't use virtual functions during
construction and destruction\n\nIt's a bad idea. Even if you think you
could use it, you will not get what you want.\n\ndynamic dispatch is
disabled during construction & destruction. These will always call the base

```

```
text_splitter = CharacterTextSplitter(chunk_size=2000, chunk_overlap=400)
texts = text_splitter.split_documents(documents)
#
```

type

```
type(texts)
```

```
list
```

```
len(texts)
```

```
13
```

```
texts[6]
```

```
Document(metadata={'source': 'OOPConcepts.pdf'}, page_content='a-
>fullThrottle();\n\n// NEW! will call Boeing747::fullThrottle\n\nb-
>fullThrottle();\n\n// NEW! will call Cessna206h::fullThrottle\n\nc-
>fullThrottle();\n\nObjects of derived classes can be used as base class
objects.\n\nPolymorphism allows us to modify the behavior of base classes
and replacing the implementation of methods. • Polymorphic methods must
be\nndeclared virtual (in C++)\n\nPolymorphism\n\n// store objects of
different types in a // datastructure using its base class Airplane **
airplanes = new Airplane*[10]; ... for(int i = 0; i < 10; i++) {\n\n// will
choose correct implementation // dynamically at runtime airplanes[i]-
>fullThrottle();\n\n}\n\nvoid foo(Airplane & a) {\n\n// this function will
work with // any class derived from Airplane\n\n}\n\nUse base class to
implement general algorithms and data structures which work with any derived
type\n\nDynamic dispatch determines the type of an object at runtime and
executes the correct method\n\nAbstract classes\n\nclass Airplane
{\n\npublic:\n\nfloat longitude, latitude;\n\nfloat heading, speed;\n\nfloat
altitude;\n\nvirtual void fullThrottle() = 0;\n\n};\n\nVirtual functions
without implementation are called pure-virtual functions. • Classes
containing pure- virtual functions can not be instantiated and are called
abstract classes\n\nTheir behavior must be defined in derived classes\n\nIs-
A vs. Has-A Relationship\n\nUse composition to manage complexity • Use
inheritance to extract common functionality • Use polymorphism to implement
general algorithms which are independent of specific types\n\nComposition =
Has-A Relationship E.g. A Boeing 747 has four jet engines.\n\nInheritance =
Is-A Relationship:\n\nE.g. A Cessna is an airplane. So is a Boeing
747.\n\nIII. Best Practices\n\nRecommendations, Design Patterns\n\nKeep your
interfaces simple, clean and consistent • Methods in a class should be as
orthogonal as possible\n\n\uf0e0 avoid method that do almost the
same\n\nMethods with the same name should do similar things • Use
encapsulation')
```

```
embeddings = HuggingFaceEmbeddings()
```

```

/tmp/ipykernel_16030/3655315981.py:1: LangChainDeprecationWarning: The class
  embeddings = HuggingFaceEmbeddings()
/tmp/ipykernel_16030/3655315981.py:1: LangChainDeprecationWarning: Default va
  embeddings = HuggingFaceEmbeddings()
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
modules.json: 100% 349/349 [00:00<00:00, 30.9kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.47kB/s]

README.md: 100% 11.6k/11.6k [00:00<00:00, 942kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.90kB/s]

config.json: 100% 571/571 [00:00<00:00, 30.8kB/s]

model.safetensors: reconstructing file: 100% 438MB / 438MB, 39.4MB/s

model.safetensors: downloading bytes: 414MB, 36.1MB/s

Loading weights: 100% 199/199 [00:00<00:00, 5484.26it/s]

tokenizer_config.json: 100% 363/363 [00:00<00:00, 22.9kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 10.1MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 28.2MB/s]

special_tokens_map.json: 100% 239/239 [00:00<00:00, 22.4kB/s]

config.json: 100% 190/190 [00:00<00:00, 16.5kB/s]

```

```
persist_directory = 'vector_db'
```

```
vector = Chroma.from_documents(documents=texts, embedding=embeddings, persis
```

```
retriever = vector.as_retriever()
#retreiver
```

```

#llm from groq
llm = ChatGroq(
    #groq_api_key=groq_api_key,
    model_name="qwen/qwen3.8-27b",
    temperature=0.7,
    max_tokens=512
)

```

```
llm
```

```

ChatGroq(metadata={'lc_versions': {'langchain-core': '1.6.1', 'langchain':
'1.3.18'}}, client=<groq.resources.chat.completions.Completions object at
0x79ce2a7be510>, async_client=
<groq.resources.chat.completions.AsyncCompletions object at 0x79cdd5ea52b0>,

```

```
model_name='qwen/qwen3.8-27b', model_kwargs={},
groq_api_key=SecretStr('*****'), max_tokens=512)
```

```
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)
```

```
query = "What are the functions mentioned in this PDF?"
response = qa_chain.invoke({"query": query})
print(response["result"])
```

Based on the provided text, the following functions (methods) are mentioned:

```
**Member Methods / Instance Methods:**
*   `Vector2D::length()`
*   `Boeing747::gearUp()`
*   `Boeing747::gearDown()`
*   `Airplane::fullThrottle()` (noted as virtual)
*   `Boeing747::fullThrottle()`
*   `Cessna206h::fullThrottle()`
*   `Engine` methods (implied by context like `fullThrottle`, though specific
*   `Gear::up()`
*   `Gear::down()`

**Constructors:**
*   `Vector2D()` (Default constructor)
*   `Vector2D(double x1, double y1)` (Parameterized constructor)
*   `Point2D(double x1, double y1)`

**Static Methods:**
*   `Point2D::reset_count()`

**Destructors:**
*   `MyVector::~~MyVector()`

**Helper/Global Functions:**
*   `sqrt()` (used inside `Vector2D::length`)
*   `foo(int param1, int param2)` (used in examples for stack/heap allocation
```

```
print(response)
```

```
{'query': 'What are the functions mentioned in this PDF?', 'result': 'Based o
```

```
print(response["source_documents"][0].metadata["source"])
```

```
OOPConcepts.pdf
```

```
# invoke the qa chain and get a response for user query
query = "Give me summary of all function from this pdf?"
response = qa_chain.invoke({"query": query})
print(response["result"])
```

```
print("***30)
print("Source Document:", response["source_documents"][0].meta
```

Based on the provided text, here is a summary of all the functions (methods)

```
### **1. `Vector2D` Class**
*   **`length()`**: A member method that calculates and returns the Euclidean
*   **`Vector2D()`**: A default constructor that initializes member variables
*   **`Vector2D(double x1, double y1)`**: A parameterized constructor that in

### **2. `Point2D` Class**
*   **`Point2D(double x1, double y1)`**: A constructor that initializes `x`,
*   **`reset_count()`**: A **static method** that resets the static member va

### **3. `MyVector` Class**
*   **`MyVector(int dim)`**: A constructor that dynamically allocates memory
*   **`~MyVector()`**: A **destructor** that frees the dynamically allocated

### **4. `Boeing747` Class**
*   **`gearUp()`**: A public method that updates physics and internally calls
*   **`gearDown()`**: A public method that updates physics and internally cal
*   **`fullThrottle()`**: A **virtual method** (in the base `Airplane` class,

### **5. `Cessna206h` Class**
*   **`fullThrottle()`**: A **virtual method** (in the base `Airplane` class,

### **6
*****
Source Document: OOPConcepts.pdf
```

```
#!pip install gradio
```

```
import gradio as gr

def rag_inference(query):
    """Performs RAG inference given a query."""
    response = qa_chain.invoke({"query": query})
    return response["result"]
```

◆ Gemini

```
iface = gr.Interface(
    fn=rag_inference,
    inputs=gr.Textbox(lines=2, placeholder="Enter your query h
    outputs="text",
    title="RAG Model Chatbot for PDF",
    description="Ask questions about the 'OOPConcepts.pdf' doc
)

# Launch the Gradio interface
# This will provide a local URL and a public shareable URL.
iface.launch(debug=True)
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires Colab notebook detected. This cell will run indefinitely so that you can see * Running on public URL: <https://8ea14fcb6e4ff0811f.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For

RAG Model Chatbot for PDF

Ask questions about the 'OOPConcepts.pdf' document and get answers based on its content.

query

Enter your query here...

Clear

Submit

output

Flag

Built with Gradio  · Settings 

Next steps:  Deploy to Cloud Run

What is Colab?

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

✓ Getting started